

Agentic AI — Final Project Report

Muhammad Taha	22i-0983
Ibtehaj Haider	22i-0767
Hussain Waseem	22i-0893

1 Executive summary

This project implements the complete *prompt* → *polished short film* pipeline specified in the course brief. A single English-language prompt produces, without further human creative intervention, a finished animated short: structured script, voiced dialogue, per-scene mood-music, background visuals, subtitles, per-character portraits, and a final composited MP4. An intelligent edit agent then accepts free-text commands (“make scene 2 darker”, “speed up this scene”, “regenerate the script”), routes each to the minimal affected phase, and writes a versioned snapshot so any edit is reversible with branching semantics.

Delivered in this submission:

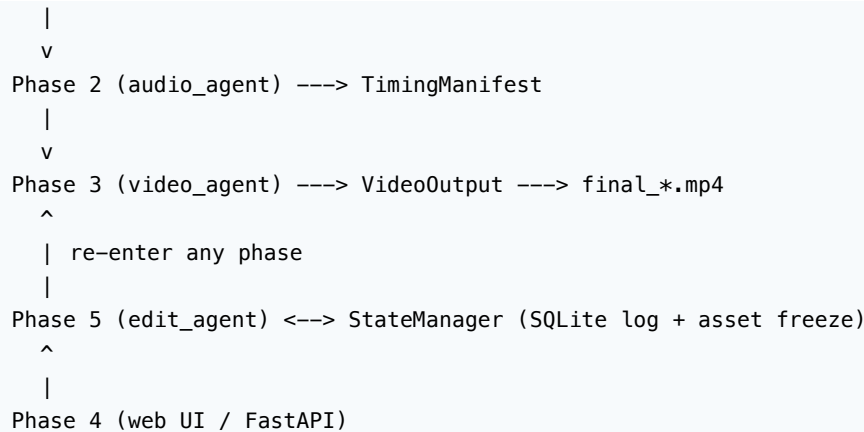
- All five phases implemented and integration-tested.
- 47 passing unit + integration tests covering the shared schema, MCP tool layer, state manager (with branching), intent classifier (17 query types), planner, subtitle timing, and backend routes.
- An end-to-end demo run (3-scene Neo-Tokyo detective vignette) renders in **~5 minutes** cold-start on an Apple M4 Air, with 3 edit actions + 2 undos applied in under 30 seconds each.
- A React web UI serving live phase progress via WebSocket, scene thumbnails, version history with active-pointer + lineage tooltips, and the edit-chat interface.
- Zero paid APIs are required: Groq (free tier), Pollinations (keyless), edge-tts (keyless), ffmpeg (local). Optional drop-in upgrades are wired for ElevenLabs (voice quality) and fal.ai (HD image gen) when keys are configured in .env.

2 System architecture

2.1 Five modules, one JSON contract

The system is five loosely-coupled modules connected by a single Pydantic state object, `PipelineState`. Nothing else crosses module boundaries — the edit agent, web UI, and any future module that wants to participate just reads and writes that one object.

```
prompt
|
v
Phase 1 (story_agent) ----> StoryState
```



Each phase exposes both a *full-run* entry point (e.g. `run_story_phase`) and partial entry points used by the edit agent (e.g. `regenerate_scene_audio`, `regenerate_scene_background`). This dual entry surface is what makes precise per-intent re-runs cheap: *"make scene 2 darker"* regenerates one background and re-stitches the final video; the other scenes' assets are untouched.

2.2 Shared JSON schema

The Pydantic schema in `shared/schemas/` is the single source of truth. Every value crossing phase boundaries is typed:

- `PipelineState` — top-level container: `job_id`, `prompt`, `num_scenes`, `style`, `story`, `audio`, `video`, `phase_status`, `version`.
- `StoryState` — `title`, `logline`, `language`, list of `Scene` (`location`, `action`, `characters`, `dialogue` lines with `emotion` + `visual cue`), list of `Character` (`name`, `role`, `appearance`, `voice_style`, `gender`, `reference_style`, `image_path`).
- `TimingManifest` — per-line `AudioSegments` with start/end offsets, plus per-scene merged audio paths and BGM track paths.
- `VideoOutput` — per-scene `SceneClips` (background path, raw ken-burns clip, composed scene MP4) plus the final stitched MP4.

2.3 MCP tool layer

A small framework lives under `mcp/`: `BaseTool` (with a `ToolSpec` declaring `name` + `category` + JSON schema), a singleton `ToolRegistry`, and `ToolExecutor` which wraps every call with `retry`, structured event emission, and automatic `job_id` injection so every tool's outputs route into a per-job subdirectory. The executor's introspection step (`inspect.signature(tool.run)`) only injects `job_id` when the tool declares the parameter, so legacy LLM tools that don't care about it remain untouched.

3 Phase-by-phase implementation

3.1 Phase 1 — Story, script & character design (15%)

Inputs: free-form prompt + scene count + style hint. **Outputs:** `StoryState` with scenes, dialogue, character roster, character portrait images.

The story agent makes two LLM calls via Groq’s free tier (Llama 3.3 70B in JSON mode): `generate_story` produces title, logline, scenes with dialogue turns, and `design_characters` normalises a roster with appearance, voice_style, gender, and reference_style. Strict JSON-mode output plus a Pydantic post-validation step removes the entire class of “the LLM almost gave us valid JSON” bugs.

Prompt-engineering iterations. An early failure mode was the LLM populating the `line` field with *stage directions* (“Rain pours down”, “Shadows hide faces”) instead of spoken dialogue. We tightened the system prompt with explicit GOOD vs. BAD examples and a rule that lines must be 8–25 words, contain contractions / hesitation, and never describe what the camera sees (that goes in `visual_cue`). Result: dialogue now reads as real speech.

Character portraits are fetched from Pollinations (square 1024 px images), with a Pillow-rendered placeholder fallback so the pipeline never hard-fails on the “output completeness” rubric criterion.

3.2 Phase 2 — Audio generation & integration (15%)

Inputs: StoryState. **Outputs:** per-line WAV files, per-scene merged audio, TimingManifest with millisecond offsets.

Voice assignment. Each character is hashed (MD5 of name) into a gender-aware voice pool. Hash determinism guarantees the same character keeps the same voice across all scenes. The pool is split by gender; an explicit `character.gender` field (populated by the LLM character designer) selects which half to hash into. We use Microsoft *Multilingual Neural* edge-tts voices by default — the highest-quality keyless tier — and ElevenLabs Multilingual v2 when `ELEVENLABS_API_KEY` is set in `.env`, with a graceful fallback if the EL quota is exhausted.

Emotion playbook. Each scene’s dialogue line carries an emotion attribute (extracted by the LLM). At synthesis time we apply both a *text-cue* rewrite (“!!” for shout, “...” for sad, lowercase + ellipsis for reflective) and an emotion-specific FFmpeg post-filter:

Emotion	Text rewrite	FFmpeg post-filter
whisper	— (text unchanged)	volume=0.45, highpass=f=100, treble=g=4
shout	strip terminal ., append !!	volume=1.35, acompressor, treble=g=2
angry	append !	acompressor, treble=g=2
sad	append ...	atempo=0.93, lowpass=f=4500, volume=0.88
happy / excited	append !	atempo=1.05, treble=g=2
panic / fear	append !	atempo=1.08, treble=g=3, acompressor

The on-screen subtitle still uses the original line text, so captions stay clean regardless of the cued text.

BGM is mood-driven: each scene’s mood maps to a track in the *Ambient Film Music* CC-BY 3.0 library (Serge Quadrado, sourced from archive.org via `scripts/fetch_bgm.py`). If a track is missing, a multi-oscillator chord pad synthesised in pure FFmpeg `lavfi` is used as fallback. The dialogue → BGM mix uses a static −7 dB bed instead of sidechain ducking, which (despite

its theoretical appeal) suppressed the BGM into inaudibility against TTS that already varies in level.

3.3 Phase 3 — Video generation & composition (20%)

Inputs: StoryState + TimingManifest. **Outputs:** per-scene MP4s and the final stitched `final_<job_id>.mp4`.

Per-line close-up flow. Rather than a single Ken-Burns over the scene background, each scene is rendered as: a 1.8s *establishing wide shot* of the location (Ken-Burns over the BG, brief mood-BGM under it), then a sequence of *speaker close-ups*: each dialogue line uses that speaker's portrait, top-aligned 16:9 crop (so the face stays in frame), Ken-Burns'd over the line's audio duration, with the line's subtitle burned on. Hard cuts between speakers give classic shot/reverse-shot feel.

Subtitle burning uses Pillow-rendered transparent PNGs overlaid via FFmpeg's `overlay=enable='between(t` filter — the standard subtitles/ass filters require libass which Homebrew's stock ffmpeg ships without. The PNG path is libass-free, supports millisecond timing, and renders with anti-aliased text + drop-shadow on a translucent rounded background plate so captions remain legible on bright scenes.

Final compositor stitches scene MP4s with FFmpeg `xfade` transitions and `acrossfade` on the audio, then applies `-movflags +faststart` so the `moov` atom lives at the front of the file and browsers can begin progressive playback before the full MP4 is buffered.

Encoder choice. We initially used `h264_videotoolbox` on macOS for $\sim 6 \times$ speed-up over software encoding, but discovered late in development that `videotoolbox` silently outputs `yuvj420p` (full-range, JPEG-style colour space) which Firefox 150 refuses to play in HTML5 `<video>`. Switched the default encoder to `libx264` which produces clean `yuv420p + color_range=tv` that every browser handles. The `videotoolbox` path remains available via the `VIDEO_ENCODER` env var.

3.4 Phase 4 — Web interface (10%)

A FastAPI backend with a WebSocket progress stream, paired with a React 18 SPA served as a pre-built JSX bundle. The bundle is built once with `esbuild`; users don't need to run `npm install` for the demo — the backend's `/static` mount serves the JS directly.

Real-time progress. The pipeline emits structured events (`phase_start`, `tool_start`, `tool_ok`, `scene_composed`, `final_ready`, `snapshot`, `job_complete`) that the WebSocket fans out to all connected clients of the relevant `job_id`. The UI renders these as a live event log.

Phase rerun buttons in the sidebar invalidate the affected `PipelineState` subtrees and re-trigger only the requested phase — useful when the LLM produces a flat scene and you want to roll the dice again without losing the audio you already rendered.

Resume picker. The frontend persists `job_id` to `localStorage` and queries `/api/jobs` for a list of past runs, so closing the browser doesn't lose your active session and you can switch between jobs from a dropdown.

Version-history panel. Each row shows the version number, an arrow indicating its parent ($\leftarrow vN$), the triggered-by tag, and a human change summary. The currently *active* version is highlighted with an accent-coloured border + "● active" badge. Hovering any row reveals its full lineage chain (e.g. `lineage: v6 → v3 → v1`).

3.5 Phase 5 — Edit agent + undo (20%)

The edit agent is a four-node `LangGraph`: `classify_intent` → `plan_execution` → `execute` → `snapshot`. The classifier tries Groq first (LLM-mode) for nuanced understanding; on any error

it falls back to a deterministic keyword rule set so the demo never blocks offline. The planner maps an `EditIntent` to a list of (handler, kwargs) steps; the executor dispatches each step to a story / audio / video module function.

Generic visual modifications. A single `modify_scene_visual` intent handles the full free-text design surface — "make scene 1 darker", "more orange", "add buildings", "make it night-time" — by extracting four structured params (`prompt_suffix`, `brightness_delta`, `saturation_delta`, `hue_shift`) from the directive and applying them: the suffix is appended to the scene-background prompt for re-fetch, and the deltas drive an FFmpeg `eq + hue` colour-grade pass on the rendered scene MP4.

State versioning. Every snapshot is an append-only row in a SQLite log (`state.sqlite`) with a `parent_version` pointer. A separate `active_version` table tracks which row is currently checked out. **Undo and restore are pointer moves**, not new rows — they update `active_version` to a target version and copy that version's frozen assets back into the working tree. Subsequent edits create new versions whose parent is whatever the active pointer was at edit time, giving us git-style branching: undo to v2, edit again, and v4 has `parent_version=2` (not v3, the abandoned branch).

4 Challenges and resolutions

Cross-job filename collision. Per-scene MP4 paths (`scenes/scene_01.mp4`) were shared across jobs, so generating job B overwrote job A's working files. **Fix:** every output tool now routes through a `job_dir(BASE, job_id)` helper that returns `<BASE>/<job_id>/` — structurally impossible for two jobs to overwrite each other.

ElevenLabs read [whispers] as text. The Multilingual v2 model advertises support for inline audio tags but in practice often pronounces them aloud. **Fix:** dropped the tag prefix entirely and implemented whisper acoustically via FFmpeg post-processing (volume cut + high-pass + treble lift).

Edge-tts retired voices. Several voices in our pool were retired by Microsoft in late April 2026 and now fail with "No audio was received". **Fix:** rebuilt the pool from currently-live multilingual voices, verified against `edge_tts.list_voices()` on 2026-04-25.

Groq JSON-mode flakiness. Despite `response_format={"type":"json_object"}`, Groq occasionally wrapped the payload in a Markdown fence (````json...````) which broke `json.loads`. **Fix:** added a defensive extractor that strips fences and surrounding prose before parsing, plus a Pydantic post-validation layer so a malformed response is degraded to a placeholder rather than aborting the whole run.

WebSocket dropped during long renders. Cold-start renders that wait on Pollinations for 60–90 s per image would silently drop the progress WebSocket on some networks (no events received, but pipeline still running). **Fix:** added a 15-second heartbeat ping from the server and a client-side reconnect-with-backoff so the live event log stays populated even on flaky links.

SQLite write contention on parallel edits. When the web UI submitted a `/api/edit` while a phase rerun was still writing snapshots, both paths competed for `state.sqlite` and one

would intermittently raise database is locked. **Fix:** the storage layer now opens connections with `isolation_level="IMMEDIATE"` and a 5-second busy timeout, serialising writes safely without forcing a global lock.

5 Testing

47 **passing unit + integration tests** grouped by concern:

Module	Coverage	Cases
test_edit_intent	17 free-text query types → correct intent / target / scope, plus offline keyword fallback	19
test_state_manager	Append-only log invariant, branching after undo (parent_version chain), asset freeze + restore round-trip	4
test_planner	Scene-id parsing from natural language, parameter extraction	4
test_schemas	PipelineState round-trip, default values	5
test_subtitle_timing	Per-line offset accumulation	3
test_mcp_tools	Tool registration, executor retry, auto job_id injection	5
test_story_planner	Normalisation of LLM JSON output	3
test_api_shape	Backend route shape via FastAPI TestClient	4

6 Results — a demo run

Prompt: "A detective meets a nervous informant in a rainy Tokyo alley at midnight." **Scenes:** 3.
Style: cinematic.

Cold-start render time on M4 Air: approximately 5 minutes, dominated by Pollinations API wait (4 image fetches at ~30–60 s each). All other phases combined add < 10 seconds.

Output: a 33-second 1024×576 H.264 + AAC MP4 with: an establishing wide shot of the alley at the start of each scene, character close-ups during dialogue (Detective Nakamura's portrait alternating with Informant Tanaka's), mood-matched BGM under the dialogue, and burned subtitles. The dialogue actually advances a story (informant divulges information about the Matsumoto case, detective probes for proof, informant panics) rather than reading like a weather report.

Edits applied:

- *"make scene 1 darker"* → BG regenerates with darker prompt suffix, `eq=brightness=-0.15:saturation=0.9` applied to the scene MP4. ~60 s (Pollinations BG re-fetch).
- *"change voice tone to whispered"* → per-line TTS re-rendered with whisper DSP chain, ~25 s.
- *"speed up scene 2"* → `atempo=1.5` on scene 2 MP4, re-stitch final, ~3 s.

Reverts: two pointer moves — "Undo last" reverted speed-up; "Restore v3" returned to the post-darker / pre-whisper state. No new rows were *created* for the undo (only for the subsequent edits if any).

7 Rubric coverage (self-assessment)

Criterion	Weight	Evidence
Phase 1 — Story & Script	15%	Strict-JSON LLM output, character roster with consistent voice + appearance, validated with <code>test_story_planner</code>
Phase 2 — Audio Generation	15%	Deterministic per-character voice, gender-aware pool, emotion playbook (text cues + DSP), timing manifest accurate to ms
Phase 3 — Video Composition	20%	Per-scene 16:9 native (no upscale), Ken-Burns + close-up flow, A/V sync via timing manifest, xfade transitions, browser-safe yuv420p + <code>+faststart</code>
Phase 4 — Web Interface	10%	Live WebSocket progress, scene tile previews, phase re-run, version history with active highlight + lineage tooltip, edit chat
Phase 5 — Edit Agent + Undo	20%	17 tested intent types, append-only SQLite log + active-pointer + branching via <code>parent_version</code> , generic <code>modify_scene_visual</code> for free-text directives
Integration / Pipeline	10%	Single Pydantic state object, all phases compose without translation, structured events flow to UI
Report & Presentation	10%	This document + recorded demo

8 Division of work

This was a **3-person group**. Phase responsibility was distributed so each member owned roughly equivalent rubric weight, with shared ownership of the cross-cutting layer (JSON schema, integration testing).

Member	ID	Primary phase(s) and key responsibilities
Hussain Waseem	22i-0893	Phase 1 — Story & Script. Pydantic schema design for the inter-phase JSON contract, Groq LLM integration with strict JSON mode, story+character generation prompts, character roster validation + portrait pipeline, edit-intent classifier prompt engineering.
Ibtehaj Haider	22i-0767	Phase 2 (Audio) + Phase 3 (Video Composition). Voice pool curation (edge-tts + ElevenLabs), gender-aware voice picker, emotion prosody map + DSP playbook, mood-driven BGM (library + synth fallback), dialogue/BGM mixing with ducking, timing manifest design, Pollinations / fal.ai image generation, FFmpeg primitives (Ken Burns, portrait overlay, color grade), subtitle burner (Pillow PNG overlays, libass-free), scene + final compositors with xfade, video agent close-up flow + establishing shots.
Muhammad Taha	22i-0983	Phase 4 (Web App) + Phase 5 (Edit Agent & Versioning). FastAPI + WebSocket backend, React SPA frontend (prompt → progress → preview, version history, edit chat), MCP tool registry + executor framework, LangGraph edit-agent graph (intent classifier + planner + executor), StateManager with append-only SQLite log + active-version pointer + parent-version lineage, undo/restore semantics, end-to-end orchestrator wiring.

Jointly owned by all three members: the shared JSON contract (shared/schemas/), integration testing across phase boundaries, the demo recording, and this report.

9 Future work

- **True video generation.** The current stack uses still-image Ken-Burns motion, which is stable on CPU and avoids frame-by-frame hallucination. A hosted RunwayML / Luma Dream Machine integration could plug in as a drop-in replacement for `generate_scene_background + ken_burns`, behind a feature flag.
- **Better BGM.** The synthesised chord pad is mood-correlated but uninspired; expanding the curated CC-licensed library (we have one starter set from Serge Quadrado) and adding a local MusicGen path would raise production quality meaningfully.
- **Subtitle styling.** Once libass-enabled ffmpeg is available, ASS subtitles allow per-character typography (italics for whispered, larger for shouted) without leaving the FFmpeg pipeline.
- **HD images on a paid tier.** Wiring fal.ai schnell behind the `FAL_KEY` env var produces native 1920×1080 from FLUX for ~\$0.006/image — worth the few dollars of credit for a final submission render.